

TRAVAUX PRATIQUES IV

Algorithmes probabilistes et d'approximation

Ce TP est à faire en utilisant le langage C. On rappelle que les options suivantes doivent être utilisées à la compilation :

```
-Wall -Wextra -fsanitize=undefined
```

A Problème des n dames : le retour

L'année dernière, nous avons introduit le problème des n dames et nous l'avons utilisé pour illustrer la méthode du retour sur trace, en OCaml. Nous allons aujourd'hui revenir sur ce problème classique et en donner une résolution probabiliste bien plus efficace en temps, en pratique. De plus, nous allons maintenant encoder ce problème en C.

A.1 Modélisation et affichage

Rappelons que le problème consiste à placer n dames sur un échiquier de taille $n \times n$ avec $n \in \mathbb{N}$, de sorte qu'aucune dame ne soit en prise (i.e. sur la même ligne, colonne ou diagonale qu'une autre dame).

Dans une solution valide au problème, on a donc exactement une dame par ligne. On représente donc une solution en stockant uniquement les positions des dames dans chacune des lignes d'indice 0 à $n - 1$, c'est à dire par un tableau $\{l_0, l_1, \dots, l_{n-1}\}$ de taille n contenant des entiers l_i compris entre 0 et $n - 1$: la position (colonne) de la dame dans la ligne i .

1. (Rappel) Dessiner la solution $\{2, 0, 3, 1\}$ sur un plateau de taille 4×4 . Est-elle une solution valide au problème des n dames pour $n = 4$?
2. Créez un fichier `dames.c` et insérez-y la fonction d'affichage `void affiche_plateau(bool** p, int n)` fournie dans le fichier `fournis.c`. Cette fonction prend en argument un plateau `p` sous la forme d'une matrice de booléens de taille $n \times n$. Une case (i, j) du damier contient une dame si `p[i][j]` contient Vrai.
 - a. Écrire une fonction `void affiche_solution(int sol[], int n)` qui prend en argument une solution au problème des n dames sous la forme d'un tableau des l_i , et affiche le plateau obtenu en utilisant la fonction `affiche_plateau`.
Vous créez un plateau sous la forme d'un tableau de booléens et le remplirez accordément. Attention à ne pas créer de VLA !!!
 - b. Tester¹ votre fonction d'affichage sur la solution $S = \{2, 0, 3, 1\}$ pour $n = 4$.

A.2 Fonctions utiles

On va remplir une solution $\{l_0, \dots, l_{n-1}\}$ au fur et à mesure, aléatoirement, en commençant à la ligne 0 jusqu'à la ligne $n - 1$, tant que c'est possible de placer une dame à la ligne suivante sans créer de conflit. Pour cela, nous aurons besoin de quelques fonctions intermédiaires utiles.

3. Lisez la fonction fournie `bool verifie_derniere(int n, int sol[], int k)` qui prend en argument une solution partielle `sol` remplie entre les lignes 0 et k incluses et vérifie que la dame placée à la ligne d'indice k n'est en conflit avec aucune dame précédente du plateau.

Générateur pseudo-aléatoire en C :

Dans ce TP, on pourra utiliser les fonctions C suivantes (et leurs exemples ci-dessous), contenues dans la bibliothèque standard `<stdlib.h>` :

- `srand(s)` : initialise le générateur pseudo-aléatoire (PRNG) avec la graine s

1. Rappelons que vous devez **toujours** tester toutes vos fonctions, même quand l'énoncé ne le demande pas explicitement...

- `rand() % n` : tire un entier entre 0 inclus et n exclus
 - `(rand() & 1) == 0` : tire un booléen
 - `(double)rand() / RAND_MAX` : tire un flottant entre 0 et 1 inclus
4. En déduire une fonction `int alea(int n, int sol[], int k)` qui prend en argument une solution partielle `sol` remplie entre les lignes 0 et k exclus (i.e. de taille k) et qui choisit aléatoirement une position sur la ligne d'indice k (entre 0 et $k - 1$) parmi les positions qui ne créent pas de conflit avec les dames précédentes. La sélection doit être uniforme sur les cases disponibles. S'il n'existe pas de telle position, on renverra l'entier -1.
- Pour faire ce tirage efficacement, vous pouvez vous inspirer de l'exercice 3 du TD : **échantillonnage**. Libre à vous, sinon, de trouver une façon de tirer uniformément une case valide dans la ligne sans appeler deux fois `verifie_derniere` sur chaque case de la ligne!*

A.3 Résolution

5. Écrire une fonction `bool descente(int n, int sol[], int k)` de type Monte Carlo qui essaie de remplir le plateau avec n dames en les positionnant aléatoirement sur chaque ligne, parmi les cases valides, tant que c'est possible (on parle d'une seule "descente" de dames). Si la fonction y parvient, alors à la fin de l'exécution le tableau `sol` contient une solution valide et la fonction renvoie `true`. Sinon, la fonction renvoie `false`, et peu importe ce qu'elle a écrit dans `sol` (par exemple, sa tentative).
6. En utilisant la fonction précédente, écrire une fonction en fonction `void sol_n_dames(int n, int sol[], int* essais)` de type Las Vegas qui trouve une solution au problème des n dames et l'écrit dans le tableau `sol`. La fonction devra également écrire dans `essais` le nombre de tentatives qui ont été nécessaires pour trouver une solution.
- Aide : Tant que la fonction précédente ne trouve pas de solution, elle refait appel à elle.*
7. Écrire une fonction `double moyenne(int n, int sol[], int nb_executions)` qui calcule le nombre moyen de descente de dames effectuées par `sol_n_dames` pour trouver une solution, sur `nb_executions` exécutions de la fonction `sol_n_dames`.
8. (bonus : révision) Écrire un algorithme qui trouve une solution au problème des n dames en temps borné, en utilisant la méthode du retour sur trace (en C).
- Je ne peux que vous encourager à faire cette fonction pour réviser! Si vous n'en avez pas le temps pendant le TP, essayez de la coder chez vous.*
9. (bonus) Comparer l'efficacité de vos fonctions de Las Vegas et de retour sur trace pour différentes valeurs de n .

B Algorithme de Freivalds

Rappelons l'algorithme de Freivalds pour vérifier qu'une matrice C donnée est bien le produit de deux matrices A et B :

Pseudo-code

```

1 Entrée : trois matrices  $A, B$  et  $C$  de tailles  $n \times p$ ,  $p \times q$  et  $n \times q$  respectivement
2 Sortie : Oui si  $AB = C$ , non sinon (avec une certaine proba d'erreur)
3 VÉRIFIER( $A, B, C$ ) :
4    $X \leftarrow$  vecteur aléatoire de  $\mathcal{M}_{q,1}(\{0,1\})$ 
5   Renvoyer  $(AB - C)X = 0$ 
```

On a montré (ou on va montrer) en TD que cet algorithme ne peut pas faire de faux négatif, et que la probabilité d'un faux positif est majoré par $\frac{1}{2}$, pour une complexité bien meilleure. Testons tout ça empiriquement!

B.1 Multiplication naïve

10. Écrire une fonction `int** mult(int** A, int n, int pA, int** B, int pB, int q)` qui prend en argument deux matrices de tailles quelconque et renvoie le produit des deux matrices (par la multiplication naïve en $\Theta(n^3)$). On fera de la programmation défensive, en vérifiant que les dimensions des matrices sont compatibles.
11. Écrire une fonction `bool egales(int**A, int**B, int n)` qui prend en argument deux matrices carrées A et B de dimensions $n \times n$ et teste l'égalité des deux matrices.
12. Écrire une fonction `void free_matrice(int** A, int p)` qui libère la mémoire allouée pour une matrice A de p lignes (et un nombre quelconque de colonnes).

13. En déduire une fonction `bool verifie_mult_naif(int** A, int** B, int**C, int n)` qui prend en argument trois matrices A , B et C carrées de dimensions $n \times n$ et teste si C est la multiplication de matrices AB . On fera attention de ne pas faire de fuite de mémoire! Vous pouvez tester avec `valgrind`.

B.2 Algorithme de Freivalds

14. (Question ouverte) En vous inspirant de la section précédente, implémentez l'algorithme de Freivalds. On écrira notamment une fonction `bool freivalds(int** A, int** B, int**C, int n)`.

Indication : vous aurez besoin de fonctions intermédiaires similaires, qui manipulent des vecteurs au lieu de matrices carrées.

15. (Bonus) Générez des "grosses" matrices, de tailles 1000×1000 , remplies avec des coefficients aléatoires entre 0 et 1000. Par exemple, dans la fonction `main`.
16. Comparez vos algorithmes naïfs et de Freivalds sur des exemples. Regardez le temps mis à répondre et la justesse des résultats.

Par exemple, vous pouvez générer aléatoirement deux grosses matrices et calculer leur produit, puis vous amuser à ajouter 1 à l'un des coefficients et voir ce que vous dit l'algorithme de Freivalds... (il suffirait de répéter n fois l'algorithme de Freivalds pour qu'il se trompe avec une probabilité $p < (\frac{1}{2})^n$).

Aide :

Vous voudrez sûrement mesurer le temps en secondes mis par une fonction précise dans votre code (on veut tester le temps de Freivalds, pas le temps de la multiplication de matrice pour créer vos tests). Pour cela, vous pouvez utiliser le code suivant :

```
1 clock_t start = clock();
2 /*Do something*/
3 clock_t end = clock();
4 float seconds = (float)(end - start) / CLOCKS_PER_SEC;
```

et afficher le temps `seconds` obtenu.